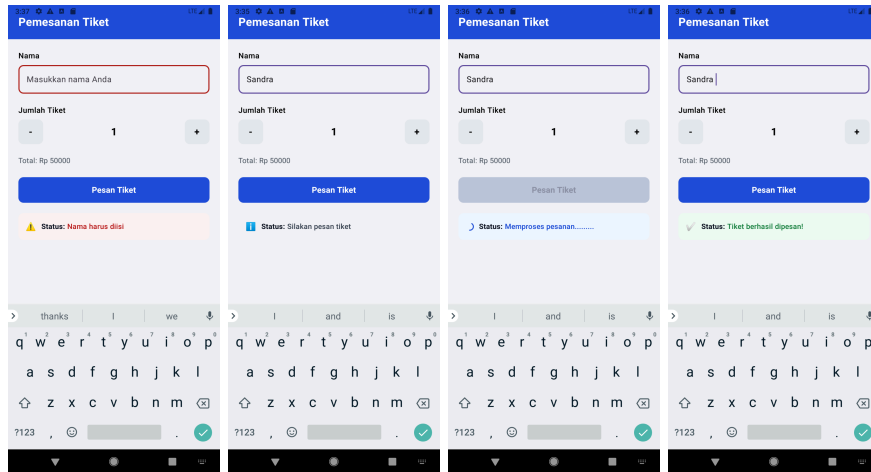


Link Github: <https://github.com/sandratриа/TemaHalamanTiket>
Screenshoot Hasil:



TUGAS MODUL 4



Oleh :

Sandra Triana Nursyafri - 245150401111027

**FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2026**

1.1.1 State Hoisting

Pra-Percobaan

1. Buat sebuah Project Android Studio baru, dengan Template **Empty Activity** untuk **Phone and Tablet**. Pastikan project yang dibuat menggunakan Jetpack Compose. Tunggu sampai proses Build inisialisasi project baru ini selesai dilakukan sebelum melanjutkan percobaan ke langkah selanjutnya.
2. Buka file `MainActivity.kt` dan tambahkan fungsi composable berikut ini di **luar** class `MainActivity`:

```
fun CounterScreen(modifier: Modifier) {  
    var number by remember { mutableStateOf(0) }  
    val label by remember { mutableStateOf("Increment") }  
    Column(modifier) {  
        Text(text = "$number", fontSize = 72.sp)  
        Button(onClick = { number++ }) {  
            Text(text = "$label")  
        }  
    }  
}
```

Import semua keyword dan konstruk atas kesalahan sintaksis dan struktur data yang muncul sebagai akibat dari penambahan kode program yang baru ditambahkan tersebut, termasuk namun tidak terbatas pada:

```
import androidx.compose.runtime.remember  
import androidx.compose.runtime.mutableStateOf  
import androidx.compose.runtime.getValue  
import androidx.compose.runtime.setValue  
import androidx.compose.material3.Button  
import androidx.compose.foundation.layout.Column  
import androidx.compose.ui.unit.dp  
import androidx.compose.ui.unit.sp
```

3. Buka file `MainActivity.kt`, temukan fungsi `onCreate()` yang telah disediakan oleh template aplikasi Android di Android Studio. Ubah bagian kode program berikut di dalam fungsi `onCreate()`:

```
Greeting(  
    name = "Android",  
    modifier = Modifier.padding(innerPadding)  
)
```

Kemudian ubah bagian tersebut menjadi:

```
CounterScreen(modifier = Modifier.padding(32.dp))
```

Sebelum menjalankan aktivitas percobaan, pastikan bahwa tidak ada penulisan kode program yang salah yang dapat mengakibatkan program tidak dapat dijalankan dengan baik pada saat di-*compile*.

Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian klik tombol "Increment" beberapa kali, ambil satu screenshot setelah tombol "Increment" diklik beberapa kali.

a. Apa yang terjadi setiap kali tombol "Increment" diklik?

Setiap kali tombol diklik, angka yang tampil bertambah 1 ($0 \rightarrow 1 \rightarrow 2 \rightarrow 3$, dst.).

b. Apa yang menyebabkan hal tersebut terjadi?

Klik tombol menjalankan `onClick = { number++ }`. Nilai state number berubah, lalu Compose melakukan recomposition pada composable yang membaca number, sehingga Text menampilkan angka terbaru.

c. Variabel *state* apa yang berperan pada Composable Text yang menampilkan perubahan angka pada Text tersebut?

Variabel state pada Text angka adalah number, yang dideklarasikan dengan `var number by remember { mutableStateOf(0) }`.

d. Variabel *state* apa yang berperan untuk menampilkan label teks pada Composable Button yang bertuliskan "Increment" tersebut?

Variabel state pada label Button adalah label, yang dideklarasikan dengan `val label by remember { mutableStateOf("Increment") }`. Nilainya tidak pernah berubah, jadi tulisannya tetap "Increment".

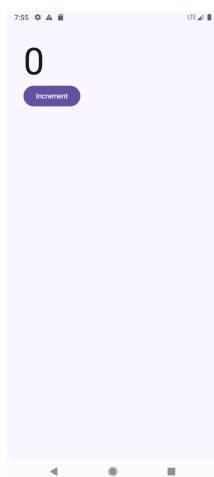
2. Pada percobaan tahap sebelumnya, ubah kode program pada Composable CounterScreen agar setiap kali tombol "Increment" diklik, nilai yang ditampilkan menjadi increment 2 (bertambah 2 setiap klik). Ubah pula label tombol "Increment" menjadi "Increment 2".

a. Perubahan apa yang perlu dilakukan dan dilakukan di mana?

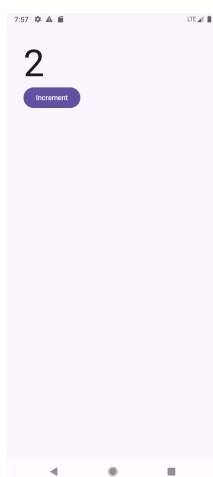
Perubahan dilakukan di dalam fungsi Composable CounterScreen. Ada dua perubahan: lambda `onClick` dari `number++` menjadi `number += 2`, dan nilai awal label dari "Increment" menjadi "Increment 2".

b. Jalankan program dan ambil masing-masing satu screenshot sebelum dan sesudah tombol Increment diklik untuk memperlihatkan bahwa perubahan tersebut berjalan dengan baik.

Gambar sebelum



Gambar sesudah



3. Langkah selanjutnya adalah melakukan *state* dan *event hoisting* pada state dan event yang sebelumnya terdapat di dalam Composable `CounterScreen`.

Ubah signature kode program pada bagian fungsi Composable `CounterScreen` dari yang sebelumnya:

```
fun CounterScreen(modifier: Modifier)
```

menjadi:

```
fun CounterScreen(modifier: Modifier,  
    number: Int, label: String, onClick: () -> Unit)
```

Ubah *body* di dalam Composable `CounterScreen` menjadi:

```
// var number by remember { mutableStateOf(0) }  
// val label by remember { mutableStateOf("Increment") }  
Column(modifier) {  
    Text(text = "$number", fontSize = 72.sp)  
    Button(onClick = onClick) {  
        Text(text = "$label")  
    }  
}
```

Ubah pemanggilan fungsi Composable `CounterScreen` di dalam fungsi `onCreate()` dari yang sebelumnya:

```
CounterScreen(modifier = Modifier.padding(32.dp))
```

menjadi:

```
var number by remember { mutableStateOf(0) }  
CounterScreen(Modifier.padding(32.dp),  
    number = number,  
    label = "Increment",  
    onClick = { number++ }  
)
```

Perhatikan bahwa deklarasi variabel state `number` dari yang sebelumnya dilakukan di dalam fungsi Composable `CounterScreen`, sekarang dideklarasikan di dalam fungsi `onCreate()` sebelum fungsi Composable `CounterScreen` dipanggil dari dalam fungsi `onCreate()`.

4. Jalankan program, dan kemudian klik beberapa kali tombol Increment sehingga angka yang ditampilkan berubah.

a. Apakah program berjalan dengan baik serta menampilkan hal yang sama persis seperti yang dilakukan pada tahap nomor 4 sebelumnya?

Ya, program berjalan sama persis seperti sebelumnya. Angka bertambah setiap klik dan label tetap "Increment".

b. Di bagian kode program manakah saat ini proses increment dilakukan?

Proses increment sekarang dilakukan di onCreate(), tepatnya di lambda onClick = { number++ } pada pemanggilan CounterScreen.

- c. Di bagian kode program manakah saat ini state Composable Text dan Composable Button "Increment" dideklarasikan?

State number dideklarasikan di onCreate() sebagai var number by remember { mutableStateOf(0) }. Label "Increment" juga ditentukan di onCreate() dan dikirim ke CounterScreen sebagai parameter. CounterScreen sekarang stateless, yaitu hanya menampilkan data dari parameter.

- 5. Diasumsikan terdapat perubahan kebutuhan fungsional aplikasi yang memaksa terjadinya perubahan logika berjalannya program pada antar muka pengguna. Ubah kode program setelah tahap nomor 7 dilakukan sedemikian rupa sehingga:

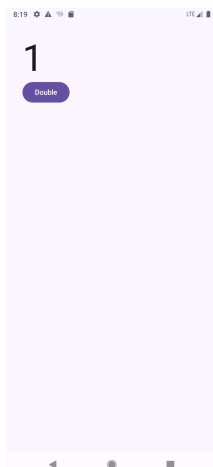
- a. Label tombol "Increment" menjadi "Double"
- b. Nilai awal variabel number dari 0 menjadi 1.
- c. Ubah fungsi/lambda onClick pada pemanggilan Composable CounterScreen di dalam fungsi onCreate() sedemikian rupa sehingga yang dilakukan dari sebelumnya: number++ menjadi menggandakan nilai variabel number.
- d. Tuliskan bagian kode program yang mendeklarasikan state dan pemanggilan fungsi Composable CounterScreen di dalam fungsi onCreate() setelah langkah nomor 8(a), 8(b), dan 8(c) dilakukan.

```
var number by remember { mutableStateOf(1) } // nilai awal 1
CounterScreen(
    Modifier.padding(32.dp),
    number = number,
    label = "Double", // label jadi Double
    onClick = { number *= 2 } // menggandakan number
)
```

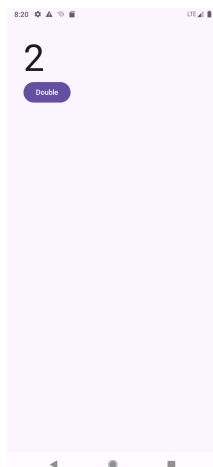
- 6. Jalankan program,

- a. Perlihatkan dengan mengambil satu screenshot sebelum dan sesudah nilai variabel number digandakan.

Gambar sebelum



Gambar Sesudah



- b. Di bagian manakah dari kode program pada percobaan tahap nomor 7 dan tahap nomor 8 dilakukan?

Semua perubahan dilakukan di dalam fungsi `onCreate()`, yaitu pada deklarasi state number (nilai awal), parameter label, dan lambda `onButtonClick`.

- c. Apakah ada perubahan yang dilakukan dari tahap nomor 7 dan tahap nomor 8 yang dilakukan di dalam fungsi `Composable CounterScreen` itu sendiri? Jika ada sebutkan perubahan yang dilakukan.

Tidak ada perubahan di dalam fungsi `Composable CounterScreen`.

7. Apa arti dari jawaban pada tahap nomor 9 terkait dengan manfaat dari melakukan *state hoisting* dalam perubahan kebutuhan fungsional aplikasi yang terkait dengan antar muka pengguna?

Karena state dan logika event sudah di-hoist ke pemanggil (`onCreate()`), perubahan kebutuhan fungsional (label, nilai awal, dan logika klik) cukup dilakukan di tempat pemanggilan tanpa menyentuh `composable CounterScreen`. Ini membuktikan bahwa `CounterScreen` menjadi komponen UI yang stateless dan reusable. Komponen yang sama bisa dipakai dengan logika berbeda (increment, double, dll.). State hoisting juga membuat komponen mudah diuji, menciptakan single source of truth, dan memisahkan logika UI dari logika state (separation of concerns). Dengan begitu risiko munculnya bug baru saat kebutuhan berubah lebih kecil.

Tugas Praktik

Lembar Kerja Praktik

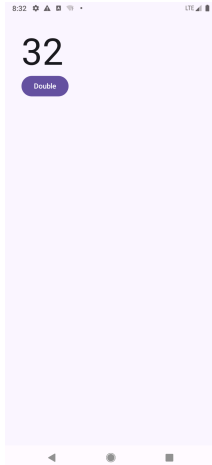
1.1.2 Perubahan Konfigurasi

Pra-Percobaan

1. Untuk percobaan terkait Perubahan Konfigurasi aplikasi Android, gunakan Project Android Studio yang sama dengan kode program percobaan *State Hoisting* setelah menyelesaikan tahap nomor 6 percobaan tersebut.
2. Pastikan program dapat berjalan dengan baik dan fungsionalitas aplikasi berjalan sesuai dengan fungsi yang didefinisikan (yaitu, mengandakan nilai variabel number).

Aktivitas Percobaan

1. Jalankan program, dengan memastikan posisi perangkat pada posisi **portrait**, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.

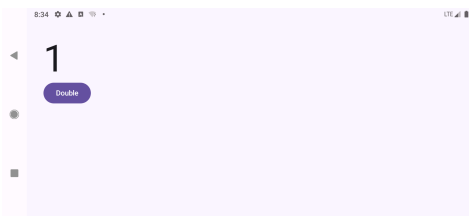


Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

Angka yang tercatat adalah 32

2. Lakukan perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.

a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Rotasi layar adalah perubahan konfigurasi yang membuat Android menghancurkan dan membuat ulang Activity. `remember` hanya mempertahankan nilai selama recomposition, bukan saat Activity dibuat ulang, sehingga state di-reset ke nilai awal (1).

Ubah bagian kode program di dalam fungsi `onCreate()` seperti berikut ini:

```
var number by remember { mutableStateOf(1) }
```

menjadi:

```
var number by rememberSaveable { mutableStateOf(1) }
```

Lakukan *import dependency* atas perubahan konstruk `remember` menjadi `rememberSaveable` yang dilakukan (dengan memposisikan kursor pada konstruk `rememberSaveable` tersebut dan melakukan refactoring dengan menekan kombinasi tombol `Alt-Enter` di jendela *editor* Android Studio).

Compile dan jalankan kembali program, dengan memastikan posisi perangkat pada posisi **portrait**, kemudian klik beberapa kali tombol "Double" sedemikian rupa sehingga angka yang ditampilkan tidak lagi angka 1 yang merupakan nilai awal yang diset pada variabel number saat program pertama kali dijalankan.

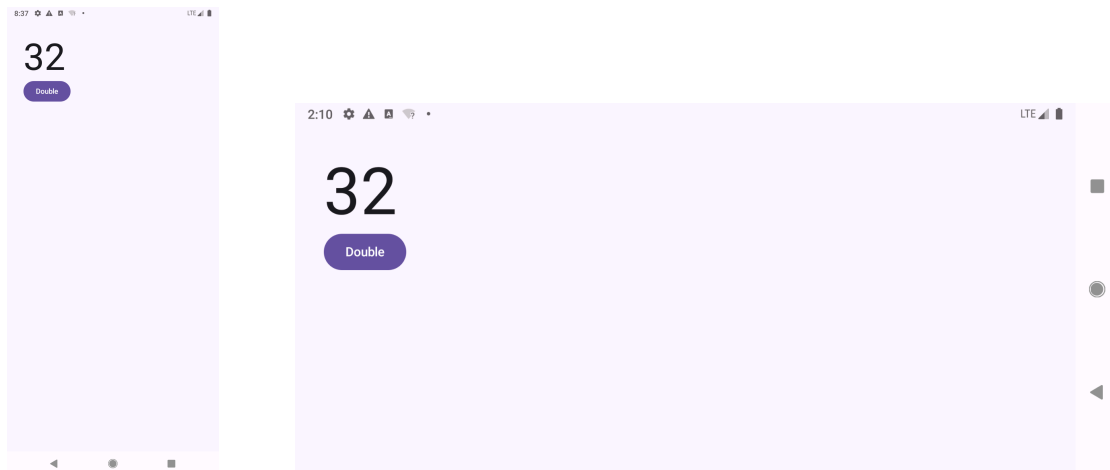
Catat angka yang ditampilkan setelah tombol "Double" diklik beberapa kali, ambil screenshot tampilan antarmukanya.

3. Lakukan kembali perubahan konfigurasi dengan memutar posisi perangkat dari posisi *portrait* menjadi *landscape*, sehingga tampilan aplikasi akan disesuaikan oleh Android.

a. Catat angka yang ditampilkan kemudian ambil screenshot tampilan antar muka aplikasi di posisi landscape.

Angka yang tampil tetap sama seperti sebelum rotasi yaitu 32, tidak kembali ke 1.

Gambar potrait Gambar landscape



b. Berapa angka yang ditampilkan? Mengapa terjadi demikian?

Karena `rememberSaveable` menyimpan nilai state ke dalam Bundle (saved instance state). Saat Activity dibuat ulang setelah rotasi, nilai itu dipulihkan.

c. Apa fungsi dari konstruk `rememberSaveable` pada kode program Android yang membedakannya dari konstruk `remember`?

Fungsi `rememberSaveable`: mempertahankan state selama recomposition dan saat Activity/proses dibuat ulang (perubahan konfigurasi seperti rotasi, atau proses yang mematikan sistem untuk menghemat memori). `remember` hanya bertahan selama recomposition dan hilang saat Activity dibuat ulang. `rememberSaveable` otomatis menyimpan tipe data yang bisa dimasukkan ke Bundle, dan untuk objek kompleks perlu `Saver custom`. Catatan: state tidak dipertahankan jika pengguna menutup aplikasi sepenuhnya, misalnya dengan swipe dari recent apps.

4. Beri contoh kasus nyata lain di mana penggunaan konstruk `rememberSaveable` ini tepat digunakan dalam aplikasi Android serta sebutkan manfaat yang diperoleh dari penerapannya!

Form login/registrasi. Teks yang diketik pengguna di `TextField` (nama, email, dll.) disimpan dengan `rememberSaveable`. Manfaatnya, saat layar berputar atau aplikasi sempat mematikan sistem, input tidak hilang. Pengguna tidak perlu mengetik ulang, sehingga pengalaman penggunaan lebih baik.

Tugas Praktik

Lembar Kerja Praktik

1.1.3 LaunchedEffect

Pra-Percobaan 1

1. Untuk percobaan ini, gunakan Project Android Studio yang sama dengan kode program percobaan Perubahan Konfigurasi setelah menyelesaikan tahap nomor 4 percobaan tersebut.
2. Pastikan program dapat berjalan dengan baik dan fungsionalitas aplikasi berjalan sesuai dengan fungsi yang didefinisikan (yaitu, menggandakan nilai variabel `number`).
3. Mengaitkan composable `SnackBar` di dalam `Scaffold`.

Di dalam fungsi `onCreate()` dan di dalam lambda `Theme{}` sebelum pemanggilan composable `Scaffold` dilakukan, tambahkan deklarasi `host state` untuk menyimpan state composable `SnackBar` yang akan digunakan untuk menampilkan pesan:

```
val snackbarHostState = remember { SnackBarHostState() }
```

Kemudian ubah bagian pemanggilan fungsi Composable `Scaffold` di dalam fungsi `onCreate()` yang sebelumnya:

```
Scaffold(modifier = Modifier.fillMaxSize())
```

menjadi:

```
Scaffold(  
    modifier = Modifier.fillMaxSize(),  
    snackbarHost = { SnackBarHost(hostState = snackbarHostState) }  
)
```

4. Menerapkan fitur Side-effect dengan `LaunchedEffect`. Dengan posisi kursor tetap berada di dalam lambda `Scaffold`, setelah pemanggilan composable `CounterScreen` tambahkan kode program *side-effect* berikut ini:

```
LaunchedEffect(number) {  
    snackbarHostState.showSnackBar("Number $number is shown!")  
}
```

5. Lakukan *import* semua *dependency* yang diperlukan dari perubahan kode program yang telah dilakukan (dengan melakukan *refactoring* melalui Alt-Enter pada bagian yang ditandai merah di dalam jendela editor Android Studio) agar perubahan kode program yang dilakukan dapat di-*compile* dan dijalankan dengan baik.

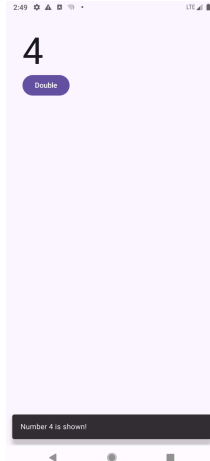
Periksa kembali perubahan kode program yang dilakukan pada pra-percobaan Side-effect ini seluruhnya dilakukan di dalam fungsi `onCreate()`. Tidak ada perubahan yang dilakukan di dalam fungsi composable `CounterScreen()`.

Aktivitas Percobaan

1. Jalankan program,

- a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?

Di bagian bawah layar muncul Snackbar bertuliskan "Number 1 is shown!". Angka di layar adalah 1.



- b. Mengapa pesan tersebut ditampilkan saat pertama kali aplikasi dijalankan tanpa ada interaksi apa pun dari pengguna?

Karena `LaunchedEffect` dijalankan otomatis saat composable pertama kali masuk ke komposisi, tanpa perlu interaksi pengguna.

- c. Apa yang memicu pesan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?

Pemicunya adalah blok `LaunchedEffect(number)` { `snackbarHostState.showSnackbar("Number $number is shown!")` } di dalam `onCreate()` (di dalam lambda `Scaffold`). Blok ini berjalan saat pertama masuk komposisi dan setiap kali number berubah.

2. Klik button "Double" satu kali,

- a. Apa yang terjadi?

Angka berubah menjadi 2 dan Snackbar baru muncul: "Number 2 is shown!".

- b. Mengapa hal tersebut terjadi? Jelaskan!

Nilai number berubah, dan number adalah key dari `LaunchedEffect(number)`. Saat key berubah, Compose membatalkan coroutine lama dan menjalankan ulang blok efek dengan nilai baru, sehingga Snackbar baru ditampilkan.

Pra-Percobaan 2

3. Untuk percobaan ini, lanjutkan Project Android Studio yang sama dengan kode program percobaan Side-effect sebelumnya.

4. Di dalam package yang sama dengan class `MainActivity`, buat sebuah class Kotlin baru bernama `Repo` (file: `Repo.kt`) dengan struktur class dan fungsi static `getData()` seperti pada kode program berikut ini:

```
class Repo {  
    companion object {
```

```

suspend fun getData() : Int {
    delay(2000) // simulating network
    return Random.nextInt(100, 1000)
}
}
}

```

Lakukan import dependency untuk fungsi `delay()` dan class `Random`. Fungsi `delay(2000)` ini digunakan untuk menyimulasikan waktu yang dibutuhkan untuk mengambil data melalui koneksi internet.

5. Di dalam fungsi `onCreate()`, setelah `LaunchedEffect` pada percobaan sebelumnya, tambahkan deklarasi fungsi `LaunchedEffect` baru berikut ini:

```

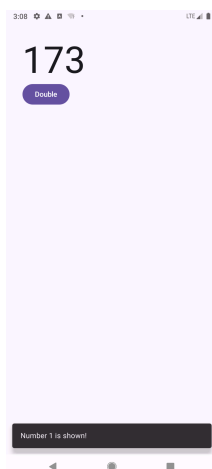
LaunchedEffect(Unit) {
    number = Repo.getData()
}

```

6. Periksa kembali perubahan kode program yang dilakukan pada pra-percobaan Side-effect ini seluruhnya dilakukan di dalam fungsi `onCreate()`. Tidak ada perubahan yang dilakukan di dalam fungsi composable `CounterScreen()`.

Aktivitas Percobaan

1. Jalankan program,
 - a. Pada saat pertama kali program dijalankan, tanpa mengklik *button* "Double" apa yang ditampilkan di dalam layar?
Saat pertama dijalankan tampil angka 1 dan Snackbar "Number 1 is shown!".
 - b. Tunggu selama dua detik, kemudian periksa kembali apa yang ditampilkan pada layar! Apa yang tampil?
Setelah 2 detik angka berubah menjadi bilangan acak antara 100 dan 999 yaitu 173, muncul Snackbar "Number 173 is shown!".



- c. Apa yang memicu perubahan angka dan pesan yang ditampilkan pada Snackbar tersebut ditampilkan? Pada bagian kode program yang mana?
Yang memicu perubahan adalah `LaunchedEffect(Unit) { number = Repo.getData() }` di

onCreate()). Blok ini berjalan sekali saat masuk komposisi, menunggu delay(2000), lalu mengubah number. Perubahan number memicu LaunchedEffect(number) berjalan lagi, sehingga Snackbar baru muncul dengan angka yang baru.

2. Dari kedua percobaan terkait dengan side-effect dengan LaunchedEffect,
 - a. Jelaskan apa perbedaan antara LaunchedEffect(state) dengan LaunchedEffect(Unit)?
LaunchedEffect(Unit) memakai key konstan yang tidak pernah berubah, sehingga efeknya hanya berjalan sekali saat composable masuk komposisi. LaunchedEffect(state) memakai state sebagai key, sehingga efeknya berjalan saat pertama masuk komposisi dan berjalan ulang setiap kali nilai state berubah (coroutine lama dibatalkan, lalu coroutine baru dimulai).
 - b. Kapan operasi di dalam LaunchedEffect(state) dengan LaunchedEffect(Unit) dijalankan?
LaunchedEffect(Unit) dijalankan hanya saat composable pertama kali masuk komposisi (misalnya untuk mengambil data saat layar dibuka). LaunchedEffect(state) dijalankan saat pertama masuk komposisi dan setiap kali nilai state berubah. Keduanya otomatis dibatalkan jika composable keluar dari komposisi.